

Tesseleringsspelet

Barchin och Charos spelar ett spel på ett rutnät som består av $2N \times 2M$ rutor. Raderna är numrerade från 0 till $2N - 1$ uppifrån och ner, och kolumnerna är numrerade från 0 till $2M - 1$ från vänster till höger. För $0 \leq i < 2N$ och $0 \leq j < 2M$ betecknar vi rutan i rad i och kolumn j med (i, j) .

Barchin ger Charos $N \cdot M$ **block**, ett efter ett. Varje block är en 2×2 kvadrat som består av fyra 1×1 **rutor**. Barchin färglägger varje ruta antingen svart eller vit, samtidigt som han ser till att **minst en ruta är vit**.

Charos måste placera varje block på rutnätet **omedelbart efter att ha mottagit det**, utan att veta vilka block hon kommer att få senare. Block kan inte roteras. Varje block måste placeras helt inuti rutnätet och täcka exakt fyra rutnätsceller. Dessutom måste den övre vänstra rutan i varje block täcka en cell vars rad- och kolumnkoordinater båda är jämna. Varje cell i rutnätet kan täckas av högst ett block.

Barchin vinner spelet om, efter att ett block har placerats, finns en 2×2 kvadrat med celler som är täckta av fyra svarta rutor. Formellt, om cellerna (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ alla är täckta av svarta rutor för någon $0 \leq a < 2N - 1$ och $0 \leq b < 2M - 1$, då vinner Barchin. Indexen a och b behöver inte vara jämna. De fyra svarta rutorna kan tillhöra samma block eller olika block.

Charos vinner om hon placerar alla $N \cdot M$ block utan att Barchin någonsin vinner. Observera att om man placerar $N \cdot M$ block täcks rutnätet helt.

Din uppgift är att ta fram en strategi som gör att Charos kan vinna spelet. Det går att bevisa att Charos, under de givna villkoren, alltid kan placera blocken på ett sätt som garanterar en seger, oavsett vilka färger blocken hon får senare.

Implementationsdetaljer

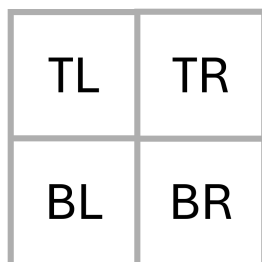
Du skall implementera följande två funktioner:

```
void init(int N, int M);
```

- N : hälften av antalet rader i rutnätet.
- M : hälften av antalet kolumner i rutnätet.
- Funktionen anropas exakt en gång per testfall, i början av programkörningen.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR);
```

- TL, TR, BL, BR : färgerna på de övre vänstra, övre högra, nedre vänstra respektive nedre högra rutorna i det aktuella blocket, enligt bilden nedan. Varje värde är antingen 0 (vitt) eller 1 (svart).
- Denna procedur anropas exakt $N \cdot M$ gånger per testfall, efter det inledande anropet till `init`.



Denna procedur ska returnera ett par heltal (i, j) , där i är radkoordinaten och j är kolumnkoordinaten för den cell där blockets övre vänstra ruta ska placeras. Både i och j måste vara jämna tal, och det område 2×2 som täcks av blocket får inte överlappa något tidigare placerat block.

Om funktionen `receive_block` någonsin returnerar ett par som inte uppfyller dessa krav, eller om ett 2×2 rutnät av celler efter att blocket har placerats blir helt täckt av svarta rutor, kommer bedömningsprogrammet omedelbart att avbryta ditt program och bedömningen för testfallet blir `Output isn't correct`.

Bedömningsprogrammets beteende är **inte adaptivt**. Detta innebär att den sekvens av block som Barchin ger till Charos är fastställd innan `init` anropas.

Begränsningar

För varje block, låt S vara antalet svarta rutor bland sina fyra rutor. Alltså, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ för varje block.

Deluppgifter

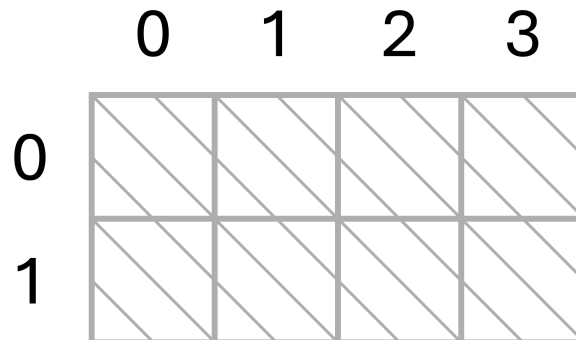
Deluppgift	Poäng	Ytterligare begränsningar
1	6	$S = 1$ för varje block, och $N = 2$.
2	16	$S = 3$ för varje block. $N = M$, N är jämnt, och var och en av de fyra möjliga blockfärgningarna förekommer exakt $\frac{N^2}{4}$ gånger.
3	10	$S = 1$ för varje block.
4	29	$S \leq 2$ för varje block.
5	39	Inga ytterligare begränsningar.

Exempel

Tänk dig ett spel där $N = 1$ och $M = 2$ gäller, vilket innebär att spelplanen har 2 rader och 4 kolumner. Gradern anropar först:

```
init(1, 2)
```

Inledningsvis är alla celler tomma. Rutnätet ser ut så här:



Det finns $N \cdot M = 2$ block att placera. Anta att Barchin ger ett block med tre svarta rutor och en vit ruta i det övre högra hörnet. Bedömaren ropar:

```
receive_block(1, 0, 1, 1)
```

Charos bestämmer sig för att placera detta block på vänster sida av rutnätet genom att returnerar $(0, 0)$.

Rutnätet ser nu ut så här:

	0	1	2	3
0				
1				

Barchin visar sedan ytterligare en blockbild med tre svarta rutor och en vit ruta i det övre vänstra hörnet:

```
receive_block(0, 1, 1, 1)
```

Den enda kvarvarande cellen med jämnt rad- och kolumnnummer som kan fungera som det övre vänstra hörnet i ett 2×2 -block är $(0, 2)$, så Charos returnerar $(0, 2)$. Rutnätet ser alltså ut så här:

	0	1	2	3
0				
1				

Ingen 2×2 kvadrat är helt täckt av svarta rutor, så Charos har lyckats placera alla block utan att Barchin någonsin har vunnit. Charos vinner spelet.

Sample Grader

Input format:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Output format:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Här är $R[k]$ och $C[k]$ paret av heltal som returneras av det k :te anropet till `receive_block`.